

Mini-ChatGPT: Завдання для груп (Етап 1)

Документ містить детальні завдання для кожної з 10 груп на етапі 1.

Загальна інформація

- **Теорія:** Теорія - MiniChatGPT — повний опис системи, формули, приклади.
- **План проєкту:** План - MiniChatGPT — архітектура, етапи, критерії готовності.
- **Структура:** кожна група (2 студенти) відповідає за один компонент.
- **Правило:** редагувати лише власну бібліотеку та її unit-тести.

Група A1 — Lib.Corpus

Роль у системі. Бібліотека завантаження та попередньої обробки текстового корпусу. Відповідає за читання файлів, нормалізацію тексту та розділення на тренувальну та валідаційну частини.

Посилання на теорію.

- Розділ 4. Корпус тексту — що таке корпус, `CorpusLoader`, `CorpusSplitter`, `CorpusLoadOptions`.
- Розділ 2. Архітектура — місце `Lib.Corpus` у графі залежностей.

Що потрібно реалізувати.

- `Corpus.cs` — `Record` з полями `TrainText`, `ValText`.
- `ICorpusLoader.cs` — інтерфейс з методом `Load(path, options)`.
- `CorpusLoader.cs` — реалізація: читання, нормалізація, розділення.
- `Configuration/CorpusLoadOptions.cs` — `lowercase`, `validationFraction`, `fallbackText`.
- `Infrastructure/IFileSystem.cs` — абстракція файлової системи (для тестів).
- `Infrastructure/DefaultFileSystem.cs` — `ReadAllText`, `Exists`.
- `Processing/CorpusTextNormalizer.cs` — опційне приведення до нижнього регістру.
- `Processing/CorpusSplitter.cs` — розділення тексту на `train/val` за `validationFraction`.

Методи.

- `Load(string path, CorpusLoadOptions? options)` — завантажити з файлу.
- `LoadFromText(string text, CorpusLoadOptions? options)` — завантажити з рядка (для тестів).
- Якщо файл відсутній — використати `fallbackText` з опцій.

Очікувана поведінка.

- Нормалізація: опційно `lowercase`.
- Розділення: наприклад 90% train, 10% val.
- Обробка відсутнього файлу: fallback-текст для інтеграційних тестів.

Критерії готовності.

- ☐ Бібліотека збирається окремо.
- ☐ Unit-тести: нормальний шлях + мінімум 2 крайні випадки (порожній текст, відсутній файл).
- ☐ `GetContractFingerprint()` оновлено та стабільне.
- ☐ README описує класи, методи та очікування інтеграції.

Група A2 — Lib.Tokenization

Роль у системі. Токенізація тексту: перетворення рядків на масиви числових ID та назад. Підтримка символного (`CharTokenizer`) та словного (`WordTokenizer`) рівнів.

Посилання на теорію.

- Розділ 5. Токенізація — `WordTokenizer`, `CharTokenizer`, побудова словника, `encode/decode`.
- Розділ 3. Контракти — `ITokenizer`.

Що потрібно реалізувати.

- `CharTokenizer.cs`, `CharTokenizerFactory.cs`, `ITokenizerFactory.cs`.
- `Model/Vocabulary.cs`, `Model/WordVocabulary.cs`.
- `Serialization/TokenizerPayloadSerializer.cs`.
- `WordTokenizer.cs` (опційно).

Методи.

- `Encode(string text) -> int[]`.
- `Decode(ReadOnlySpan<int> tokens) -> string`.
- `BuildFromText(string text)`.
- `FromPayload(JsonElement payload)`.
- `GetPayloadForCheckpoint()`.

Очікувана поведінка.

- Зворотність: $\text{Decode}(\text{Encode}(\text{text})) \approx \text{text}$ (з урахуванням unknown-токенів).
- Невідомі токени: стратегія UNK (наприклад, ID 0).
- Для `WordTokenizer`: нормалізація регістру та пунктуації.

Критерії готовності.

- ☐ Unit-тести: encode/decode round-trip, unknown tokens, порожній текст.
- ☐ Сумісність з форматом чекпоінту (розділ 9).

Група A3 — Lib.Batching

Роль у системі. Формування батчів пар (контекст, ціль) для тренування. Вибір випадкових вікон з потоку токенів.

Посилання на теорію.

- Розділ 10. Тренування — потік тренування, пари (context, target).
- Розділ 12. Потік даних.

Що потрібно реалізувати.

- `Batch.cs`, `IBatchProvider.cs`, `ITokenStream.cs`, `TokenBatchProvider.cs`.
- `Configuration/BatchConfig.cs`.
- `Streams/ArrayTokenStream.cs`, `Sampling/BatchWindowSampler.cs`.

Метод. `GetBatch(int batchSize, int blockSize, Random? rng)` — повернути батч пар (context, target).

Очікувана поведінка.

- Валідні вікна: не виходити за межі потоку.
- Короткий вхід: edge cases (менше токенів, ніж `blockSize`).
- Детермінованість: при фіксованому seed — однакові результати.

Критерії готовності.

- ☐ Unit-тести: нормальний батч, короткий потік, детермінованість.

Група A4 — Lib.MathCore

Роль у системі. Математичні примітиви: softmax, cross-entropy loss, argmax, семплінг з дискретного розподілу.

Посилання на теорію.

- Розділ 7. Математичні операції — Softmax, Cross-Entropy, формули.
- Розділ 6.3. TinyNN — використання Softmax у тренуванні.

Що потрібно реалізувати.

- `IMathOps.cs`, `MathOps.cs`, `MathOpsImpl.cs`.
- `Calculators/SoftmaxCalculator.cs`, `Calculators/LossCalculator.cs`.
- `Utilities/ScoreUtilities.cs`, `Sampling/ProbabilitySampler.cs`.
- `MatrixOps.cs`, `VectorOps.cs` (опційно).

Методи.

- `Softmax(ReadOnlySpan<float> logits) -> float[]`.
- `CrossEntropyLoss(ReadOnlySpan<float> logits, int target) -> float`.
- `ArgMax(ReadOnlySpan<float> scores) -> int`.
- `SampleFromProbs(ReadOnlySpan<float> probs, Random rng) -> int`.

Очікувана поведінка.

- Softmax: числова стабільність (віднімання $\max$ перед $\exp$).
- Сума ймовірностей після softmax = 1.
- Тестованість через інтерфейс `IMathOps`.

Критерії готовності.

- Unit-тести: softmax сума=1, cross-entropy для відомих розподілів, argmax.

Група A5 — Lib.Sampling

Роль у системі. Вибір наступного токена при генерації: temperature scaling, top-k обрізання, нормалізація.

Посилання на теорію.

- Розділ 8. Семплінг — Temperature, Top-K, алгоритм Sampler.
- Розділ 11. Чат та генерація — використання в `ModelTextGenerator`.

Що потрібно реалізувати.

- `ISampler.cs`, `Sampler.cs`, `SamplingOptions.cs` (опційно).
- `Processing/TemperatureScaler.cs`,
`Processing/TopKSelector.cs`,
`Processing/ProbabilityNormalizer.cs`.
- `NucleusSampler.cs` (опційно).

Методи.

- `Sample(float[] probs, float temperature, int topK, Random? rng) -> int`.
- Перевантаження з `int? seed` для детермінованості.

Очікувана поведінка.

- $T < 1$: різкіший розподіл, $T > 1$: м'якший.
- Тор-К: лише K найкращих токенів мають ненульову ймовірність.
- Детермінованість при фіксованому seed.

Критерії готовності.

- ☐ Unit-тести: temperature вплив, top-k обрізання, seed.

Група А6 — Lib.Models.NGram

Роль у системі. N-грамні мовні моделі: bigram (1 попередній токен) та trigram (2 попередні). Ймовірності на основі підрахунку в корпусі.

Посилання на теорію.

- Розділ 6.1. Bigram — формули, навчання, NextTokenScores.
- Розділ 6.2. Trigram — fallback на біграму.
- Розділ 9. Чекпоінт — modelPayload для ngram.

Що потрібно реалізувати.

- NGramModel.cs, TrigramModel.cs.
- NGramModelFactory.cs, INGramModelFactory.cs.
- Statistics/NGramCounts.cs, Metrics/PerplexityCalculator.cs, Serialization/NGramPayloadMapper.cs.

Методи.

- Train(ReadOnlySpan<int> tokens).
- NextTokenScores(ReadOnlySpan<int> context) -> float[].
- FromPayload(JsonElement), GetPayloadForCheckpoint().
- ComputePerplexity(ReadOnlySpan<int> tokens) (опційно).

Очікувана поведінка.

- Bigram: $P(next|prev) = \frac{count(prev,next)}{\sum count(prev,*)}$.
- Trigram: якщо є (p2,p1) — використати; інакше — bigram по p1.
- Сумісність з чекпоінтом.

Критерії готовності.

- ☐ Bigram працює, trigram — stretch.
- ☐ Perplexity для валідації.
- ☐ Checkpoint round-trip.

Група А7 — Lib.Models.TinyNN

Роль у системі. Нейронна мовна модель: embedding + linear head. Тренується градієнтним спуском.

Посилання на теорію.

- Розділ 6.3. TinyNN — архітектура, `EmbeddingLayer`, `LinearHead`, forward/backward, `TrainStep`.
- Розділ 7. Математичні операції — Softmax для loss.
- Розділ 9. Чекпоінт — `embeddings`, `outputWeights`, `outputBias`.

Що потрібно реалізувати.

- `TinyNNModel.cs`, `Configuration/TinyNNConfig.cs`.
- `State/TinyNNWeights.cs`.
- `Layers/EmbeddingLayer.cs`, `Layers/LinearHead.cs`.
- `Factories/TinyNNModelFactory.cs`.

Методи.

- `EncodeContext(context)` — усереднення ембеддингів контексту.
- `Project(hidden)` — лінійна проекція на vocab.
- `NextTokenScores(context)` — forward pass.
- `TrainStep(context, target, lr)` — forward, loss, backward, SGD.

Очікувана поведінка.

- `ContextSize`: обрізати контекст до останніх N tokenів.
- Ініціалізація: малі випадкові ваги (наприклад, scale 0.1).
- Checkpoint: зберігати та відновлювати всі ваги.

Критерії готовності.

- ☐ Forward pass дає коректні логіти.
- ☐ `TrainStep` зменшує loss.
- ☐ Checkpoint round-trip.

Група A8 — Lib.Models.TinyTransformer

Роль у системі. Міні-трансформер: один decoder block (self-attention + feed-forward).

Посилання на теорію.

- Розділ 6.4. TinyTransformer — SelfAttentionLayer, FeedForwardLayer, каузальна маска, Q/K/V.
- Розділ 9. Чекпоінт — modelPayload для tinytransformer.

Що потрібно реалізувати.

- TinyTransformerModel.cs,
Configuration/TinyTransformerConfig.cs,
State/TinyTransformerWeights.cs.
- Layers/SelfAttentionLayer.cs,
Layers/FeedForwardLayer.cs,
Layers/TransformerBlock.cs.
- Factories/TinyTransformerModelFactory.cs.

Методи.

- Compute(context, embeddingSize) — causal self-attention, hidden останньої позиції.
- Project(hidden, vocabSize) — FFN + output projection.
- Forward(context, vocabSize, embeddingSize) — attention → FFN → logits.
- NextTokenScores(context) — з урахуванням ContextSize.

Очікувана поведінка.

- Каузальна маска: позиція i бачить лише $0..i$.
- Scale: $\text{scores} / \sqrt{d}$ перед softmax.
- Можна почати зі scaffold (без тренування), головне — компіляція і тести.

Критерії готовності.

- ☐ Forward pass повертає логіти.
- ☐ Checkpoint round-trip.
- ☐ (Стретч) Backward pass для тренування.

Група A9 — Lib.Training

Роль у системі. Абстракція циклу тренування: епохи, кроки, чекпоінти, метрики. Оркеструє модель та batch provider.

Посилання на теорію.

- Розділ 10. Тренування — потік тренування, `TrainTinyNN`.
- Розділ 12. Потік даних.

Що потрібно реалізувати.

- `ITrainingLoop.cs`, `TrainingLoop.cs`, `TrainingLoopImpl.cs`.
- `TrainingConfig.cs`.
- `Scheduling/CheckpointScheduler.cs`, `Metrics/TrainingMetrics.cs`.

Метод. `Train(ILanguageModel model, IBatchProvider batchProvider, TrainingConfig config)`.

Очікувана поведінка.

- Епохи: прохід по всіх батчах.
- Checkpoint cadence: наприклад, кожні N епох.
- Метрики: середній loss за епоху.

Критерії готовності.

- ☐ Цикл тренування працює з NGram.
- ☐ (Стретч) Підтримка TinyNN через `TrainStep`.

Група A10 — Lib.ChatConsole

Роль у системі. Інтерактивний REPL чату: введення промпту, генерація відповіді, команди (`/reset`, `/temp`, тощо).

Посилання на теорію.

- Розділ 11. Чат та генерація тексту — `ModelTextGenerator`, REPL, команди.
- Розділ 3. Контракти — `ITextGenerator`.

Що потрібно реалізувати.

- `ChatRepl.cs`, `ReplOptions.cs`.
- `Commands/CommandExecutionContext.cs`,
`Commands/IReplCommand.cs`,
`Commands/CommandRegistry.cs`.
- `Commands/HelpCommand.cs`, `ResetCommand.cs`, `SeedCommand.cs`, `TempCommand.cs`,
`TopKCommand.cs`, `QuitCommand.cs`.

Методи.

- `Run(float temp, int topK, int seed)` — головний цикл.
- Обробка команд: якщо рядок починається з `/` — виконати команду, інакше — генерація.

Очікувана поведінка.

- REPL приймає введення, передає в `ITextGenerator`, виводить результат.
- Команди змінюють контекст (`temp`, `topK`, `seed`) або виконують дії (`reset`, `quit`).
- Зручний консольний UX: привітання, підказки.

Критерії готовності.

- ☐ REPL працює з будь-якою моделлю (model-agnostic).
- ☐ Усі команди реалізовані.
- ☐ Unit-тести для команд.

Загальні критерії завершення етапу 1

Для кожної групи:

- **Збірка:** бібліотека збирається незалежно.
- **Тести:** unit-тести покривають нормальний шлях + мінімум 2 крайні випадки.
- **Fingerprint:** `GetContractFingerprint()` оновлено та стабільне.
- **README:** опис класів, методів та очікувань інтеграції для етапу 2.

Швидкі посилання на теорію

Розділ	Посилання
Вступ, архітектура	§1–2
Контракти	§3
Корпус	§4
Токенізація	§5
Моделі (Bigram, Trigram, TinyNN, TinyTransformer)	§6
Математика	§7
Семплінг	§8
Чекпоінт	§9
Тренування	§10
Чат	§11
Потік даних	§12